

Lab-13 Take Home
ID1063: Introduction to Programming
2024 - Semester 1
Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad
Due Date: 4th November 2024, 11:59pm

The Lab-13 take-home assignment consists of three tasks focused on file input-output operations. You can download the codebase and all relevant files from the classroom announcement. Template code for each task is located in the root folder within their respective directories—Task-1, Task-2, and Task-3.

A shell script “**compile_run.sh**” for each task is also provided for automatic testing and execution of your code (placed inside the respective tasks folder), assuming you are using a Linux machine. If you’re working on a Windows system, you can either set up WSL-2 or install the Cygwin environment to enable Linux functionality. Instructions for configuring these environments are provided in the Lab-1 PPT (accessible here: https://iith-my.sharepoint.com/:p:/g/personal/saurabhkr_cse_iith_ac_in/EbQGFnehB19EnMcfAH6bWGoBVdACLeC2ngZTgpuxzf03GQ?e=LzjS8L), starting from slide 26.

Please reach out to the Instructors/TAs if you encounter any issues setting up the Linux environment on Windows. We now begin with the detailed descriptions of each task for this assignment followed by the next sections.

1 Expense Manager [10 Points]

You are creating a program called **Expense Manager** to help users keep track of their expenses. The program should allow users to input the expense name, date, and amount, saving each entry in `expenses.txt`. Users can view all recorded expenses and calculate their monthly total.

1.1 Task

Write a C program that performs the following:

- **Add a new expense.**
 - Ask for the expense name, date (YYYY-MM-DD), and amount.
 - Save the information to `expenses.txt`.
- **View all expenses.**
 - Use `fscanf()` and `feof()` to read expenses from the file and display them.
- **Calculate and display the total expense amount.**
- **Allow the user to reset the expense list**, clearing all data from `expenses.txt`.
- Use `rewind()` after reading the expenses list to allow continued updates if needed.

- Close the file using `fclose()` after the file operation.

Note: Please navigate to the **Task-1** directory and use the file **task1.c** for the implementation of this task.

1.2 Input Format

- **Add expense:** Expense Name, Date, Amount
- **View expenses:** command prompt

1.3 Output Format

- Display expense list and show the total amount spent.

1.4 Example

Input:

```
Option: Add
Enter Expense Name: Grocery
Enter Date: 2023-10-16
Enter Amount: 150
Option: View
```

Output:

```
Expense List:
1. Name: Grocery, Date: 2023-10-16, Amount: 150
Total Expenses: 150
```

2 Chronicle of the Forgotten Spells [10 Points]

In the magical realm of Eldoria, a young sorcerer named Alaric has been entrusted with the Chronicle of Forgotten Spells. This ancient tome holds the knowledge of powerful spells, but it can only be written at the end, and spells can only be erased from the end. Additionally, Alaric can reverse the most recent modifications, either restoring spells he accidentally erased or removing those he recently added. However, he can only redo an undone modification once.

Your task is to help Alaric keep track of his modifications to the Chronicle as he adds, removes, undoes, or redoes his changes.

2.1 Task

You need to implement a program that will process a sequence of operations and modify the Chronicle accordingly. The operations are as follows:

1. **INSERT x :** Append the spell x to the end of the Chronicle.
2. **DELETE k :** Erase the last k characters from the Chronicle. It's guaranteed that k won't exceed the current Chronicle length.
3. **Revert the last INSERT or DELETE operation.**
4. **Reapply the last undone operation, if any.**

At the end of the sequence, print the final state of the Chronicle in a file called `output.txt`. If the chronicle is empty, print `EMPTY`.

Note: Please navigate to the **Task-2** directory and use the file **task2.c** for the implementation of this task.

2.2 Input Format

There are T test cases provided to you via files. Your code needs to read the value of T first. The i^{th} test case is provided in the file with filename `input_i.txt`, in the `input` directory. Each such file contains input data in the following format:

- The first line contains an integer N , the number of operations.
- Each of the following N lines can contain any operation out of the following four: (1) INSERT x (2) DELETE k (3) UNDO (4) REDO

2.3 Output Format

Consequently, for the i^{th} input file, print the output in a file with the output filename `output_i.txt` in a directory named `solution`, containing the final state of the Chronicle. If the Chronicle is empty, print EMPTY.

2.4 Example

2.4.1 Input File

```
7
INSERT Fireball
INSERT Heal
DELETE 4
UNDO
REDO
DELETE 2
UNDO
```

2.4.2 Output File

```
FireballHeal
```

NOTE: The output must be displayed inside a file named `output_i.txt` where i is the i^{th} test case.

2.4.3 Explanation

The operations unfold as follows:

```
INSERT Fireball ==> "Fireball"
INSERT Heal ==> "FireballHeal"
DELETE 4 ==> "Fireball" (removed "Heal")
UNDO ==> "FireballHeal" (restores deleted "Heal")
REDO ==> "Fireball" (reapplies delete of "Heal")
DELETE 2 ==> "Fireba"
UNDO ==> "Fireball" (restores deleted "ll")
```

3 Weird Ctrl F [10 Points]

You are given a dictionary of words $\mathcal{D}$ and a separate list of keywords $\mathcal{L}$.

3.1 Task

For each keyword k in $\mathcal{L}$, find and print all words in the dictionary $\mathcal{D}$ such that at least one substring of a dictionary word appears as a subsequence in k . Sort these words in descending order based on their string similarity score, which can be computed using one of 8 possible scoring methods:

The similarity score between a dictionary word $d \in \mathcal{D}$ and a keyword $k \in \mathcal{L}$ is determined using one of two general approaches:

1. Using character counts: $\text{sim}(d, k) = \sum_{c \in a, b, \dots, z} \text{charCount}_d(c) \cdot \text{charCount}_k(c)$.
2. Using substring and subsequence counts: $\text{sim}(d, k) = \sum_{s \in \text{all distinct substrings of } k} \text{substrCount}_d(s) \cdot \text{subseqCount}_k(s)$.

The specific scoring function to be used is defined by one of the following 8 variations:

- $\text{sim}_0(d, k)$: $\text{charCount}_d(c)$ and $\text{charCount}_k(c)$ return 1 if character c is present in d or k , respectively, and 0 otherwise.
- $\text{sim}_1(d, k)$: $\text{charCount}_d(c)$ returns the count of occurrences of character c in d , while $\text{charCount}_k(c)$ returns 1 if c is present in k , and 0 otherwise.
- $\text{sim}_2(d, k)$: $\text{charCount}_d(c)$ returns 1 if c is present in d , while $\text{charCount}_k(c)$ returns the count of occurrences of c in k .
- $\text{sim}_3(d, k)$: Both $\text{charCount}_d(c)$ and $\text{charCount}_k(c)$ return the count of occurrences of c in d and k , respectively.
- $\text{sim}_4(d, k)$: $\text{substrCount}_d(s)$ returns 1 if s is a subsequence of d , and 0 otherwise, while $\text{subseqCount}_k(s)$ returns 1 if s is a substring of k , and 0 otherwise.
- $\text{sim}_5(d, k)$: $\text{substrCount}_d(s)$ returns 1 if s is a subsequence of d , while $\text{subseqCount}_k(s)$ returns the count of occurrences of s as a substring in k .
- $\text{sim}_6(d, k)$: $\text{substrCount}_d(s)$ returns the count of occurrences of s as a subsequence in d , while $\text{subseqCount}_k(s)$ returns 1 if s is a substring of k , and 0 otherwise.
- $\text{sim}_7(d, k)$: $\text{substrCount}_d(s)$ returns the count of occurrences of s as a subsequence in d , and $\text{subseqCount}_k(s)$ returns the count of occurrences of s as a substring in k .

In case of ties in similarity scores, sort the matching words in lexicographically descending order.

Note: Please navigate to the **Task-3** directory and use the file **task3.c** for the implementation of this task.

3.2 Input Format

There are T test cases provided to you via files. Your code needs to read the value of T first. The i^{th} testcase is provided in the file with filename `input_i.txt`, in the `input` directory. Each such file contains input data in the following format:

```
|D|
D[0]
D[1]
:
D[|D| - 1]
|L|
L[0] f0
```

$$\begin{aligned} &\mathcal{L}[1] \ f_1 \\ &\vdots \\ &\mathcal{L}[|\mathcal{L}| - 1] \ f_{|\mathcal{L}| - 1} \end{aligned}$$

3.3 Output Format

Consequently, for the i^{th} input file, print the output in a file with the output filename `output_i.txt` in a directory named `solution`, in the following format:

```
cnt0
str[0][0]
str[0][1]
⋮
str[0][cnt0 - 1]
cnt1
str[1][0]
str[1][1]
⋮
str[1][cnt1 - 1]
⋮
⋮
cnt|\mathcal{L}| - 1
str[|\mathcal{L}| - 1][0]
str[|\mathcal{L}| - 1][1]
⋮
str[|\mathcal{L}| - 1][cnt|\mathcal{L}| - 1 - 1]
```

In the above output, cnt_i denotes the number of dictionary words that matched keyword i . The next cnt_i lines print the corresponding dictionary words in descending order, using the sorting comparator sim_{f_i} (as mentioned earlier, if there are ties, we use the lexicographical order).

3.4 Notes

- $1 \leq |\mathcal{D}|, |\mathcal{L}| \leq 100$.
- Each word in $\mathcal{D}$ consists of lowercase English letters, with lengths between 2 and 25 characters.
- Each word in $\mathcal{L}$ consists of lowercase English letters with lengths between 2 and 6 characters.
- $0 \leq f_i \leq 7$ for all $0 \leq i \leq |\mathcal{L}| - 1$, where each f_i indicates the similarity function to be used when comparing $\mathcal{L}_i$ with words in $\mathcal{D}$.
- Substrings and subsequences considered for comparison must be at least two characters long.
- A subsequence of a string s is a subset of characters from s , which obey the order in which they appear in s .
- A substring of s is a contiguous subsequence of s .
- The visible testcases are provided under the `input` and `output` directories, respectively. Your code should read from all the input test cases and print the output for each test case in the prescribed

format under the `solution` directory. The shell script provided will then check if the output your code prints under `solution/output_i.txt` matches `output/output_i.txt`.

- As indicated earlier, your code must read the number of test cases from **stdin** first, followed by reading input from the input files.

3.5 Example

3.5.1 Input File

```
3
sun
water
earth
2
hearty 0
sweut 7
```

3.5.2 Output File

```
2
earth
water
1
sun
```

3.5.3 Explanation

We examine the first keyword, 'hearty'. hearty has a subsequence eart which is one of the substrings of the dictionary word 'earth'. hearty does not have any subsequence of length at least two that exists as a substring in sun. However hearty has a subsequence er which exists as a substring of the dictionary word water. Thus, earth and water are two candidate strings that match with hearty. To calculate the order in which they need to be printed, we need to use $\text{sim}_0(\cdot, \cdot)$ function. $\text{sim}_0(\text{hearty}, \text{earth}) = 5$ and $\text{sim}_0(\text{hearty}, \text{water}) = 4$. Accordingly, we print earth first and then water.

Similarly, for the keyword sweut, only one dictionary word sun matches it, so only that is printed (the subsequence is su, that exists as a substring of sun).

Deliverable:

1. Source code of problems 1, 2, and 3 which you have implemented. Document your code properly. The source code of each problem should be placed in the same place as provided in the code base. **Please do not change Lab-13 Take Home code structure as we will be using automated scripts for testing.**
2. Feel free to discuss the assignment among your friends, instructor, and TAs. However, at the end of the day, we want you to implement by yourself.
3. Submit the **tarball/zip** file of your assignment named as **your roll number**. For example, if your roll number is **xy24btechXXXXX** then your submission file **tarball/zip** file name should be either **xy24btechXXXXX.zip** or **xy24btechXXXXX.tar**

Note: We will accept your submission via **classroom** only, and any other submission mode is strictly prohibited such as submitting via email.